

Experiment No. 2

Title: Implementing an interface

Theory:

In Java, an interface is a abstract class that defines a contract or a blueprint for other classes to follow. It specifies a set of methods that must be implemented by any class that implements it. An interface is essentially a way to define a common set of methods that can be called by other classes, without caring about how those methods are implemented.

Key Characteristics of Interfaces

Here are the key characteristics of interfaces in Java:

1. **Abstract:** Interfaces are abstract, meaning they cannot be instantiated on their own.
2. **No Implementation:** Interfaces do not provide any implementation for the methods they declare.
3. **Multiple Inheritance:** A class can implement multiple interfaces, which allows for multiple inheritance.
4. **Public and Abstract Methods:** All methods in an interface are public and abstract, meaning they must be implemented by any class that implements the interface.
5. **No State:** Interfaces do not have any state, meaning they do not have any instance variables.

Since Java 8, we can have **default and static methods** in an interface.

Why we use java interface:

There are mainly three reasons to use interface. They are given below.

- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.
- It can be used to achieve loose coupling.

Declaring an interface:

An interface is declared by using the interface keyword. It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default. A class that implements an interface must implement all the methods declared in the interface.

Syntax:

```
interface <interface_name>{
```

```
// declare constant fields
// declare methods that abstract
// by default.
}
```

How classical problem is solved using interface in java:

The diamond problem is a well-known issue in object-oriented programming that arises when a class inherits from two classes that have a common base class. In Java, this problem can be solved using interfaces. Here's an example:

// The common base class (Grandparent)

```
class A {

    void show() {

        System.out.println("A's show()");

    }

}
```

// The first intermediate class (Parent 1)

```
class B extends A {

    @Override

    void show() {

        System.out.println("B's show()");

    }

}
```

// The second intermediate class (Parent 2)

```
class C extends A {

    @Override
```

```
void show() {  
    System.out.println("C's show()");  
}  
}
```

// The class that inherits from both B and C (Child)

```
class D extends B, C {  
    // No implementation needed here  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        D d = new D();  
        d.show(); // Output: B's show()  
    }  
}
```

In this example, class D inherits from both B and C, which both inherit from A. This creates a diamond-shaped inheritance graph, hence the name "diamond problem".

The issue here is that D inherits two different implementations of the show() method, one from B and one from C. Which one should be called when we invoke d.show()?

To solve this problem, we can use interfaces instead of classes for the intermediate layers. Here's the modified code:

// The common base interface (Grandparent)

```
interface A {  
    void show();  
}
```

// The first intermediate interface (Parent 1)

interface B extends A {

// No implementation needed here

}

// The second intermediate interface (Parent 2)

interface C extends A {

// No implementation needed here

}

// The class that implements both B and C (Child)

class D implements B, C {

@Override

public void show() {

System.out.println("D's show()");

}

}

class Main {

public static void main(String[] args) {

D d = new D();

d.show(); // Output: D's show()

}

}

By using interfaces instead of classes, we avoid the diamond problem altogether. The D class implements both B and C interfaces, and provides its own implementation of the show() method.

Note that in this example, we've moved the implementation of the show() method from the intermediate interfaces to the D class itself. This is because interfaces cannot provide implementation, only declare methods.

By using interfaces, we've decoupled the inheritance graph and avoided the diamond problem.